Milestone report

MILESTONE: Beta
TEAM NAME: Noflex
GAME NAME: Q
NAME: Minjae Kyung
JOBS: Product Manager
DEGREE PROGRAM: RTIS
GAME DESCRIPTION: Our game is a side view platformer rogue-like game. Player is an eminent scientist who participated in the space project.
LINKS: https://github.com/Rudy-Castan-DigiPen-Teaching/game-project-noflex

Work Done :

- *More Enemies*
  - We need more enemies. So I designed 4 normal enemies and 1 new boss.
    - Engineer



      - If he found player he shoot bullet and run to other side
      - If he reached the other side. He spawn Turret and dead(balance thing)
      - Player should kill him before he spawn Turret
    - Turret



      - If he found player, He shoot many bullet
      - Turret is very strong
    - Bomber



      - If he found a player, it would follow and self-destruct.

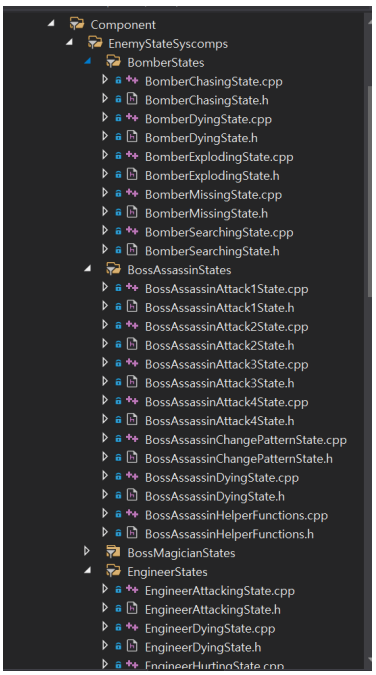
- Flamethrower

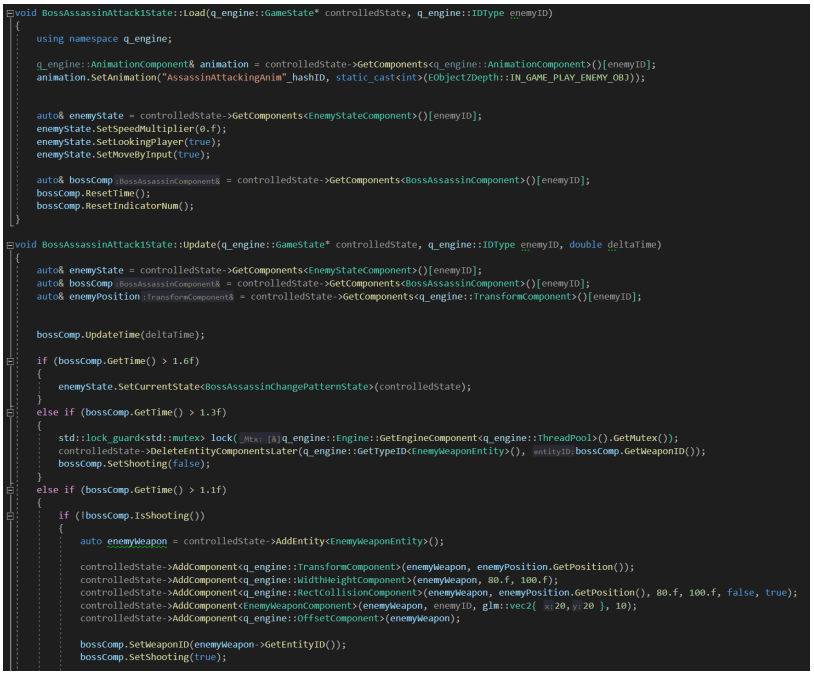


  - If the player reaches flamethrower attack range, It flames.
- Boss Assassin



  - Pattern 1 : It teleports to behind the player and attack it twice
  - Pattern 2 : It teleports to player and attack from above
  - Pattern 3,4 : It moving and attack to player





```
void BossAssassinAttack1State::Load(q_engine::GameState* controlledState, q_engine::IDType enemyID)
{
    using namespace q_engine;

    q_engine::AnimationComponent& animation = controlledState->GetComponents<q_engine::AnimationComponent>()[enemyID];
    animation.SetAnimation("AssassinAttackingAnim"_hashID, static_cast<int>(EObjectZDepth::IN_GAME_PLAY_ENEMY_OBJ));


    auto& enemyState = controlledState->GetComponents<EnemyStateComponent>()[enemyID];
    enemyState.SetSpeedMultiplier(0.f);
    enemyState.SetLookingPlayer(true);
    enemyState.SetMoveByInput(true);

    auto& bossComp :BossAssassinComponent& = controlledState->GetComponents<BossAssassinComponent>()[enemyID];
    bossComp.ResetTime();
    bossComp.ResetIndicatorNum();
}

void BossAssassinAttack1State::Update(q_engine::GameState* controlledState, q_engine::IDType enemyID, double deltaTime)
{
    auto& enemyState = controlledState->GetComponents<EnemyStateComponent>()[enemyID];
    auto& bossComp :BossAssassinComponent& = controlledState->GetComponents<BossAssassinComponent>()[enemyID];
    auto& enemyPosition :TransformComponent& = controlledState->GetComponents<q_engine::TransformComponent>()[enemyID];

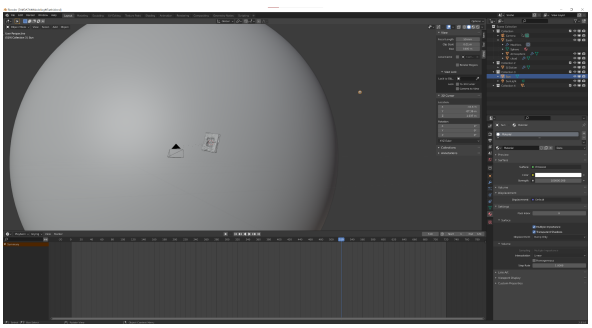

    bossComp.UpdateTime(deltaTime);

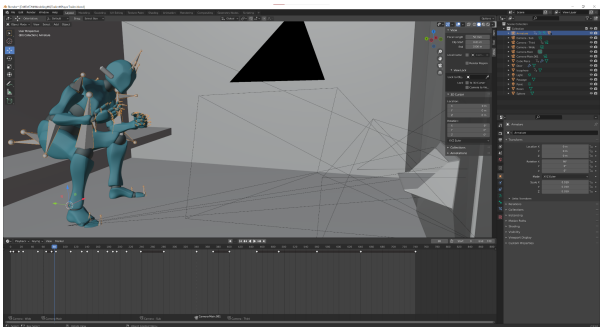
    if (bossComp.GetTime() > 1.6f)
    {
        enemyState.SetCurrentState<BossAssassinChangePatternState>(controlledState);
    }
    else if (bossComp.GetTime() > 1.3f)
    {
        std::lock_guard<std::mutex> lock( _Mtx: [&]q_engine::Engine::GetEngineComponent<q_engine::ThreadPool>().GetMutex());
        controlledState->DeleteEntityComponentsLater(q_engine::GetTypeID<EnemyWeaponEntity>(), entityID:bossComp.GetWeaponID());
        bossComp.SetShooting(false);
    }
    else if (bossComp.GetTime() > 1.1f)
    {
        if (!bossComp.IsShooting())
        {
            auto enemyWeapon = controlledState->AddEntity<EnemyWeaponEntity>();

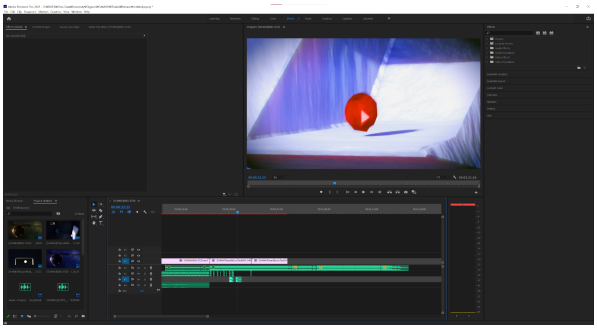
            controlledState->AddComponent<q_engine::TransformComponent>(enemyWeapon, enemyPosition.GetPosition());
            controlledState->AddComponent<q_engine::WidthHeightComponent>(enemyWeapon, 80.f, 100.f);
            controlledState->AddComponent<q_engine::RectCollisionComponent>(enemyWeapon, enemyPosition.GetPosition(), 80.f, 100.f, false, true);
            controlledState->AddComponent<EnemyWeaponComponent>(enemyWeapon, enemyID, glm::vec2{ x:20,y:20 }, 10);
            controlledState->AddComponent<q_engine::OffsetComponent>(enemyWeapon);

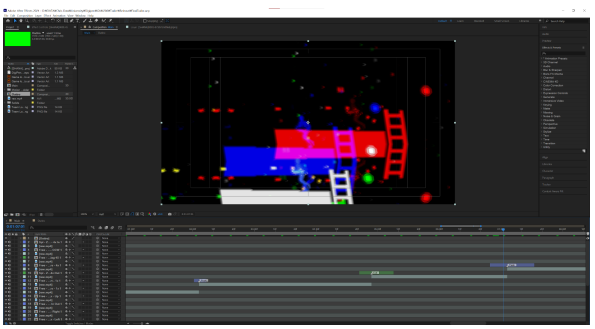
            bossComp.SetWeaponID(enemyWeapon->GetEntityID());
            bossComp.SetShooting(true);
```

- Trailer
  - I made a trailer for our game.
  - Player model was from Mixamo and other models were made by me.
  - And I animated it.









Notes :